```
}
```

Clearly this approach does not scale well. Generics solve this problem by allowing us to substitute type parameters for these types. Now users of the Pair struct must provide the types that the Pair will hold (i.e.,the user instantiates the generic with particular types).

```
public partial struct Pair<T, U>
{
 public T Fst { get; private set; }
 public U Snd { get; private set; }
 public Pair(T fst, U snd) : this()
 {
      Fst = fst;
      Snd = snd;
 }
  public static void Test()
 {
  Pair<int, string> p = new Pair<int, string>(0, "hi");
}
}
```

Classes, interfaces, methods, and delegates can all be declared generic. In particular, you can declare a method to be generic inside of a generic class if you need to mention additional type parameters in the method.

```
public static class NestedGenerics<T>
{
     public static void GenericMethod<U>(T t, U u) { }
}
```

One particular use of generic methods is to be able to write a method over a generic class, say a collection. You do not wish to instantiate the type variable of the collection because your method should work for all possible collections. Generics methods let us tackle this problem.

```
public static class GenericMethod
{
      public static void ProcessCollection<T>(ICollectio
n<T> c) { }
}
```

4.2 Generic Constraints

Inside of a generic declaration, you don't know what types will be used